

# CAMPUS QUEUE

## Presentation flow and rehearsal runbook

Practical demonstration sequence

Prepared for project demonstration, code review and academic viva.

8 October 2026

Verified source, practical SQL and honest implementation boundaries. Credentials are deliberately excluded.

# Contents

|                                                          |   |
|----------------------------------------------------------|---|
| The presentation goal                                    | 2 |
| Before the judge arrives                                 | 3 |
| Sequence 1: introduce the problem - 60 seconds           | 3 |
| Sequence 2: landing page and signup - 90 seconds         | 3 |
| Sequence 3: preview before confirmation - 90 seconds     | 4 |
| Sequence 4: staff perspective - 2 minutes                | 4 |
| Sequence 5: missed student and cancellation - 90 seconds | 4 |
| Sequence 6: estimates, lunch and closing - 90 seconds    | 4 |
| Sequence 7: architecture - 60 seconds                    | 5 |
| Sequence 8: frontend code walkthrough - 2 minutes        | 5 |
| Sequence 9: backend code walkthrough - 3 minutes         | 5 |
| Sequence 10: database demonstration - 3 minutes          | 6 |
| Sequence 11: CRUD practical response                     | 6 |
| Sequence 12: deployment proof and closing                | 6 |
| Team roles and handoffs                                  | 6 |
| Rehearsal questions to ask each other                    | 7 |
| Recovery plan during a live presentation                 | 7 |
| Final confidence checklist                               | 8 |
| Academic demonstration edition   8 October 2026          |   |

## The presentation goal

Make the judge understand the problem, then prove one student request moves through the interface, Java logic and database. Do not begin by opening arbitrary code. Introduce the workflow first so code questions have context.

Suggested allocation: 12-15 minutes for the application and architecture, then 10-15 minutes for code/database questions. Shorten the UI tour if the judge wants to inspect files early. The master guide is your technical reference; this document is your presentation order.

## Before the judge arrives

Open the student site, staff site, backend catalogue, repository and Campus Queue Cloud Workbench connection. Render Free can sleep; warm the service by opening the real API shortly before presenting. Confirm actual operating time. The normal schedule is 10 AM-1 PM and 2 PM-3 PM, with lunch 1-2 PM.

Use two browser profiles or separate windows for student and staff. Have the three staff account emails available privately, but keep passwords off presentation slides and shared PDFs. Use a clearly labelled demo student. Do not display database credentials or password hashes.

Run a read-only SELECT on services and verify assignments in staff/counters. Prepare `campus_queue_viva` for CRUD practice. Keep a backup screenshot or short recording for network failure, labelled as backup evidence rather than live output.

## Sequence 1: introduce the problem - 60 seconds

Say: "Students often stand in a physical line at campus offices without knowing when their turn will come. Our application gives them a digital place in the queue and gives staff a controlled way to process it."

Name the three services. Explain the student can reserve a same-day place before opening, review an estimate, and return to the desk when called. State the scope: queue management, not online payment processing or automatic document verification.

Transition: "I will show a student's booking, then the staff action, and finally where the same record appears in the database and code."

## Sequence 2: landing page and signup - 90 seconds

Open <https://campusqueue.netlify.app/>. Point to the three-step workflow and published counter hours. Choose Sign up. Explain name/email/mobile/password validation. If creating a fresh demo account, use a labelled synthetic account and a syntactically valid test number, not another person's number.

Explain: "Client validation helps the user, but Java validates again because requests can bypass the UI." Password minimum is six characters with a letter and number. Do not claim OTP verification. Sign in and show that refresh retains a valid session.

## Sequence 3: preview before confirmation - 90 seconds

On overview, select General Inquiries. Explain clicking a card only opens the booking flow. Show position, people ahead and estimated service clock time. Mention that confirmed order can change between preview and submission.

Say: "The token is created only when I confirm Reserve my place. Position ten means nine people ahead, not necessarily token ten." Confirm a reservation, then open My queue and Bookings. Show that the same token/service/status appears.

If before 10 AM, show the estimate begins at 10 AM. If after 3 PM, show booking closure and explain the real rule rather than changing it mid-demo. Use backup test evidence for unavailable transitions.

## Sequence 4: staff perspective - 2 minutes

Open <https://campusquestaff.netlify.app/> and sign into the corresponding service account. Point to the service and counter assignment, student's name, email and phone icon. Explain that all three staff accounts use the same website, but the backend limits operations to their assigned service.

During open hours: summon the first student, show CALLED, Start service, then Complete. Switch to the student window and refresh/wait for polling. Show the result. With another waiting student, show automatic next assignment during the same operating session.

Explain the tel: icon without making an actual call to a test number: "This opens the device's calling application; it does not change the queue state." Do not demonstrate a real telephone call without the student's permission.

## Sequence 5: missed student and cancellation - 90 seconds

Explain first absence returns the ticket behind up to ten waiting students; second absence ends that ticket. With only one or two students, the ticket goes behind the available students, so the visual position will not always be ten or eleven.

Use controlled demo tickets if showing these actions; do not mark an unrelated live student absent. Alternatively show the local test evidence and the corresponding ReservationService branch. Demonstrate cancellation on your own waiting ticket and explain why a serving ticket cannot be cancelled through that path.

## Sequence 6: estimates, lunch and closing - 90 seconds

Say: "Our estimate uses recent actual processing durations. Booking at 7 AM does not add three hours to a task completed at 10:03. We count service-start to completion within open windows."

Give the arithmetic: five completed tasks total fifteen minutes, so average is three. Ten people ahead implies about thirty minutes with one active counter. Explain this is a forecast; it updates with completed samples and does not continuously estimate an unusually long task's remaining duration.

Explain lunch is skipped in clock projections. At closing, WAITING tickets roll to the next day's 10 AM queue with token/order/misses preserved. They are not completed. No weekend/holiday calendar is implemented.

## Sequence 7: architecture - 60 seconds

Describe browser -> Netlify /api proxy -> Render/Tomcat -> JDBC -> Railway MySQL. Workbench is another client of Railway. GitHub stores and deploys code; it does not automatically synchronize a local database.

Mention Java 17, Servlet 6, Tomcat 10.1, MySQL, React and Vite. Explain the backend is a WAR deployed as ROOT.war, not Spring Boot. Retained JSPs are legacy pages, while the demonstrated UI is React.

## Sequence 8: frontend code walkthrough - 2 minutes

Open QueueSystemFrontend/src/main.jsx, then App.jsx. Identify HashRouter and the route to bookings. Find BookingForm. Explain useState/useEffect, the preview fetch, controlled form inputs and confirmation handler. Open student-api.js to show form encoding, credentials, timeout, JSON parse and error handling.

For staff, open frontend/src/pages/StaffDashboard.jsx and services/api.js. Show polling, busy/error handling and the completion action. Explain UI state updates from API responses; it does not directly connect to MySQL.

If a random CSS line is selected, name its selector, what property changes, and which screen uses it. Distinguish layout changes from business logic. Use the master guide's numbered source appendix if needed.

## Sequence 9: backend code walkthrough - 3 minutes

Open StudentPortalServlet.java at /bookings/create. Explain route mapping, authenticated student ID and date checks. Then open ReservationService.reserve. Walk through service lock, preview validation, next token/order, queue insertion, metadata/event insertion, commit and rollback.

Open ReservationService.operate for staff ownership/state verification. Explain CALLED -> SERVING -> COMPLETED. Open WaitingTimeService to show recent twenty sample SQL, then ServiceHours.processingMinutes for the processing-window intersection.

Open DBConnection.java last, without revealing environment secrets. Explain configuration names and DriverManager. Show Passwords.java for hashing and StudentValidation.java for server rules if asked. Explain input/output and failure behaviour rather than reciting syntax.

## Sequence 10: database demonstration - 3 minutes

In Workbench use Campus Queue Cloud. Expand campus\_queue. Show students without password columns, services, queue, queue\_reservations and queue\_events. Explain legacy bookings is not the current reservation table.

```
SELECT student_id, student_name, email FROM campus_queue.students;
SELECT service_id, service_name FROM campus_queue.services;
SELECT q.queue_id, q.token_number, q.status, s.student_name,
       r.visit_date, r.queue_order
FROM campus_queue.queue q
JOIN campus_queue.students s ON s.student_id=q.student_id
JOIN campus_queue.queue_reservations r ON r.queue_id=q.queue_id
ORDER BY q.queue_id DESC LIMIT 10;
```

Point out the exact queue record you just created. Explain primary keys, foreign keys and joins. Refresh after a staff/student action to prove the websites and Workbench use the same cloud database.

## Sequence 11: CRUD practical response

If asked to insert/delete/update, say: "I will use a scratch table so we do not alter other students' live records." Create demo\_visits in campus\_queue\_viva, insert one labelled row, use LAST\_INSERT\_ID, update by primary key, SELECT the changed row, then DELETE by that key. The full executable exercise is in the master guide.

If asked to demonstrate transaction rollback, insert a scratch row inside START TRANSACTION, SELECT it, ROLLBACK, and SELECT again. Explain that DDL and DML have different transaction implications in MySQL.

If the judge insists on an application-table operation, explain the required relationships first and obtain the exact intended test scope. Do not directly set a live queue to COMPLETED merely to simulate business logic; the normal service action also records processing and events.

## Sequence 12: deployment proof and closing

Show GitHub main, Render Live, the API's three services and the two Netlify projects. Explain the MySQL public TCP port is separate from Tomcat HTTP 8080. Describe the Netlify proxy and its routing fallback. Give the free-tier cost limitations honestly.

Close with: "The project connects an online reservation to controlled staff processing and a database history. We have tested key rules locally and verified the cloud connections and login flows. Future work includes an admin portal, stronger account verification, holiday scheduling and more detailed ETA modelling."

Invite code/database questions. Do not finish with a claim that every edge case or security concern is solved.

## Team roles and handoffs

Presenter 1: problem, landing page, student reservation and ticket. Presenter 2: staff workflow, absence policy and service assignments. Presenter 3: Java transaction/ETA and backend request flow. Presenter 4: database SQL, deployment and evidence. If the team has fewer members, combine adjacent roles.

Each person should know the architecture and token/state explanation. Handoff phrases: "This confirmed request now reaches our Java service; my teammate will show the transaction." "This database row is the same ticket you saw on both dashboards."

Do not divide ownership by old branch names alone. All final components are integrated on main. Pull the latest main before rehearsing, and inspect TEAM-CHANGELOG.md and CLOUD-DEPLOYMENT.md together.

## Rehearsal questions to ask each other

1. What happens if two students reserve at exactly the same time?
2. Why is a token different from queue position?
3. Where does the server obtain the current student's identity?
4. Can Fee Payment staff process a General Inquiries ticket?
5. Why does a 7 AM booking not inflate processing average?
6. Which table receives current bookings?
7. What is the first-miss insertion position with only three waiting students?
8. What remains after a rollback?
9. Why can the backend root return 404 while the API works?
10. Why does Workbench show cloud data without local/cloud offline sync?
11. What would you change to add an administrator portal?
12. What is still a known limitation in future-ticket cancellation?

## Recovery plan during a live presentation

Slow first load: explain Render's idle sleep and allow startup; do not repeatedly create bookings while waiting. API returns HTML: verify the /api proxy rule precedes the SPA fallback. Catalogue error: check Render safe SQLState logs and compare the current public database endpoint. Login fails: verify account provisioning and proxy session cookie before changing passwords.

Outside hours: demonstrate preview/prebooking/closure and show local state-transition tests; do not secretly enable a production test clock. Internet failure: use labelled backup screenshots and local code/SQL scratch practice, explaining that cloud access is unavailable. Duplicate booking rejection: show it as successful validation, not a system crash.

Wrong database port: retain Render PORT=8080 but use the Railway public port inside QUEUE\_DB\_URL. An empty old response can be cached; use a fresh request and inspect the live commit. Never paste a password onto the projector to speed up troubleshooting.

## Final confidence checklist

Can every presenter explain one random frontend line, one Java SQL-binding line, and one database relationship? Can they draw the request path without notes? Can they calculate position versus people ahead? Can they distinguish completed-processing samples from configured defaults? Can they admit limitations without confusion?

Have the PDFs, source repository, test results, live URLs and safe SQL exercise ready. Share private demo login details separately. After rehearsal, cancel unused waiting tickets and leave a clean demonstration queue.